Backend (main) Code Review

Nov 7, 2026 · @Sajawal

Summary

Verdict: the backend on `main` does not start and its tests do not run. The individual modules are mostly well written, but they were merged without ever being run together. `main.py` is empty, three modules that other files import do not exist, and the swap feature calls a column name that isn't in the model.

Item	State
Code reviewed	`main` at `2f61ea7`, `backend/` + `docs/`, about 5,300 lines of Python
App starts?	No. `backend/app/main.py` is 0 bytes
Test suite as committed	Fails at import: `ModuleNotFoundError: No module named 'app.config'`
Tests with my throwaway stubs (scratch copy only)	116 tests: 112 pass, 4 fail on real logic bugs
Untested modules	auth, swaps, notifications, conflict checker, permissions: 0 tests
Issues found	7 critical, 9 major, 9 moderate, 10 minor

How I checked: read every backend file, statically checked every `from app...` import, installed `requirements.txt` in a scratch venv and ran `pytest`. To get past the import errors I added stub `config.py`, `dependencies.py` and `get_current_user` in a **scratch copy only**. No repo files were changed.

Critical

Each of these stops the app from starting or makes a whole feature crash. All were confirmed by running the code.

#	Issue	Where	Evidence / impact	How to fix
C1	`main.py` is empty	`backend/app/main.py` (0 bytes)	There is no `FastAPI()` app, no routers included, no CORS, no table creation. `uvicorn app.main:app` fails. Nothing is reachable.	Create the app: `app = FastAPI(title="Timetable Automation")`, add `CORSMiddleware` with the frontend origin from settings, and `app.include_router(...)` for all 13 routers. Add `GET /api/health`. One person owns this file so routers aren't added in conflicting spots.
C2	`app/config.py` does not exist	imported by `core/security.py:8`, `database.py:5`	First import of either file raises `ModuleNotFoundError: No module named 'app.config'`. This is why the whole test suite fails before running a single test.	Add `config.py` with a `pydantic-settings` `Settings` class: `DATABASE_URL`, `JWT_SECRET_KEY` (no default, must come from `.env`), `JWT_ALGORITHM="HS256"`, `ACCESS_TOKEN_EXPIRE_MINUTES`, `UPLOAD_DIR`, `MAX_UPLOAD_MB`, `CORS_ORIGINS`. Commit a `.env.example` with placeholder values.
C3	Authentication dependency missing	`core/security.py`, `app/dependencies.py`; used in `permissions.py:7`, `auth.py:26-27`, `makeup.py:8`, `notifications.py:5`, `portal_deps.py:11`, `conftest.py:19`	`get_current_user` is imported from two different places and defined in neither. `app/dependencies.py` doesn't exist. `DUMMY_HASH` is imported but not defined. Every protected route fails to import, so there is no working auth at all.	Create `app/dependencies.py` with `get_current_user`: read the `Authorization: Bearer` header via `HTTPBearer`, call `decode_access_token`, load the `User` by `sub`, return 401 if missing, expired or inactive. Define `DUMMY_HASH = hash_password("timing-dummy")` in `security.py`. Then make every file import `get_current_user` from `app.dependencies` only.
C4	Wrong import path in 12 files	`models/event.py`, `exam_paper.py`, `grade_record.py`, `lecture_material.py`, `software_resource.py`; `routers/rooms.py`, `gpa.py`, `materials.py`, `events.py`, `software.py`, `exams.py`, `timetable.py`	They use `from database import ...` instead of `from app.database import ...`. Raises `ModuleNotFoundError: No module named 'database'` when run from `backend/`.	Change to `from app.database import Base` / `get_db` in all 12. Add a `pytest.ini` (or `pyproject.toml`) in `backend/` with `pythonpath = .` so tests import the same way the app does.

#	Issue	Where	Evidence / impact	How to fix
C5	`auth.py` doesn't match the `User` model	`routers/auth.py` vs `models/user.py`	`auth.py` queries and writes `User.identifier`; the model has no such column (it has `email`). `activate` expects roster students with `password_hash = None`, but the column is NOT NULL. Register, login, activate and create-user all crash.	Decide one login key (the code and docs say roll number). Add `identifier: Mapped[str] = mapped_column(String(50), unique=True, index=True)`, make `email` optional, and make `password_hash` nullable. Then write `test_auth.py` covering register, login, wrong password, activate twice and non-admin creating staff.
C6	Swaps and conflict checks use a column that doesn't exist	`services/conflict_checker.py:21,66,79,95`, `services/swap_service.py:88-89,171`, `core/permissions.py:30`	Code reads `TimetableEntry.day`; the column is `day_of_week`. `can_request_swap` uses `entry.teacher.user_id`, but `TimetableEntry` has no `teacher` relationship. Every swap request returns 500.	Rename to `day_of_week` everywhere (the makeup solver already uses it). Add `teacher = relationship("Teacher")` to `TimetableEntry`, or load the `Teacher` by `entry.teacher_id` in `can_request_swap`. Add `test_swaps_api.py` and `test_conflict_checker.py`.
C7	Code was merged without ever being run	process / repo settings	C1 to C6 would each have failed in the first `pytest` run. That means PRs were approved by reading only, with no CI, and the guide's "one backend + one frontend reviewer" rule didn't catch it.	Add `.github/workflows/backend-tests.yml`: install `requirements.txt`, run `pytest backend/test`. Make that check required on `main` in the ruleset. Reviewers must run the branch locally before approving.

Major

M1 to M3 are the 4 tests that still fail once the import errors are stubbed out.

#	Issue	Where	Evidence / impact	How to fix
M1	Events update uses PATCH; contract and test say PUT	`routers/events.py:98` vs `docs/api-contract-wajiha.md:121`, `test_events_api.py:17`	Test fails with `KeyError: 'title'` (405 response). Rooms and software use PUT; events alone uses PATCH. Frontend built from the contract will get 405.	Change to `@router.put(...)` to match the contract and the other routers. If partial update is wanted, keep PUT with an all-optional body, as rooms do.

#	Issue	Where	Evidence / impact	How to fix
M2	`instructor_name` is ignored for teachers	`routers/exams.py:47-50`, `routers/materials.py:51-54`	A teacher uploading for a colleague always gets their own name stored. Breaks search by instructor (`test_exam_search_and_filters`) and grouping (`test_materials_grouped_by_instructor`). Code and tests disagree on the rule.	Decide the rule and write it in the contract. Most likely: use the given name if provided, otherwise the uploader's. Then make code and tests match, and add one test for each case.
M3	Makeup solver puts lecture courses in labs	`services/scheduler_rules.py:127`	The rule only blocks lab courses from lecture rooms, not the reverse. The solver recommends `Lab-1` for a lecture course (`test_makeup_solver_room_fallback_and_booking`). Labs are the scarcest rooms, so they get used up.	Make it a two-way match: lecture courses need `room_type == "lecture"` (hard rule), or keep labs allowed but rank them last (soft rule). Choose one, then update the test name to say which.
M4	Makeup booking has no ownership check or lock	`routers/makeup.py` `book_makeup_class`	Any staff user can book a makeup for any `teacher_id`. Two bookings for the same room and slot can both pass the clash check and commit (no lock, unlike swaps).	Teachers may book only for their own `Teacher` row; HOD/admin for their department. Lock the room and section rows with `with_for_update()` before the re-check, as `swap_service` does. Add a DB unique constraint on (room, date, start time) as a last line of defence.
M5	Two rule engines that disagree	`services/conflict_checker.py` and `services/scheduler_rules.py`	Swaps use `conflict_checker`, makeups use `scheduler_rules`. `conflict_checker` never looks at `MakeupClass`, so an approved swap can collide with a booked makeup. Every rule change has to be made twice.	Make `scheduler_rules` the single source of rules. Have `conflict_checker` call it and include `MakeupClass` rows in its "existing slots". Add one test where a swap clashes with a makeup and must be rejected.
M6	Four different role-check implementations	`core/permissions.py`, `routers/portal_deps.py`, `routers/makeup.py:29`, inline in `routers/auth.py`	Different error messages, and different handling of Enum vs string roles. `portal_deps.py` itself says to replace it with `permissions.py` once merged; that never happened.	Keep `core/permissions.py` `require_roles` only. Change each router to `from app.core.permissions import require_roles`, delete `portal_deps.require_roles` and `_require_staff`, keep `STAFF` / `ADMINS` tuples in `permissions.py`.

#	Issue	Where	Evidence / impact	How to fix
M7	No database migrations	whole backend	Nothing creates tables (no `create_all`, no Alembic). The team plan uses MySQL, so schema changes will be applied by hand and drift between machines.	Add Alembic: `alembic init`, point `env.py` at `Base.metadata` and `settings.DATABASE_URL`, generate the first migration from current models, and document `alembic upgrade head` in the README.
M8	Open self-registration contradicts the design	`routers/auth.py:13-16` vs `:119`	The docstring says students claim pre-imported accounts and there is no public register. But `/register` lets anyone with a roll-number-shaped string create a student in any section. The code comment admits it can't prove the roll number is theirs.	Pick one path. Recommended: delete `/register`, keep `/activate` for roster students, and add a one-time activation code from the roster import. If `/register` stays, require the roll number to exist in the roster.
M9	No single API contract	`docs/api-contract-wajiha.md`	The file itself says to paste it into `docs/api-contract.md`; that wasn't done. Auth, swaps, notifications and makeup have no contract at all. The frontend already guessed wrong (email vs roll number).	Create `docs/api-contract.md` with every endpoint, method, body and response, including auth. Merge Wajiha's section in. Make "contract updated" part of the PR checklist. FastAPI's `/docs` page can be used to check it.

Moderate

#	Issue	Where	Evidence / impact	How to fix
D1	Riskiest modules have zero tests	auth, swaps, notifications, permissions, conflict checker	The modules with locking, state changes (approve/reject) and security have no tests, while simple CRUD has many. This is how C5 and C6 got in.	Add `test_auth.py`, `test_swaps_api.py`, `test_notifications_api.py`, `test_permissions.py`, `test_conflict_checker.py`. Minimum: one happy path, one permission denial and one invalid state per endpoint.
D2	HOD accounts can never approve swaps	`auth.py` `create_staff_user` + `permissions.py:can_decide_swap`	`/users` takes no `department_id`, so every HOD is created with `department_id = None`. `can_decide_swap` then returns False for every swap.	Add `department_id` to `CreateUserRequest`, require it when `role == "hod"`, and check the department exists. Add a test: HOD of dept A approves A's swap, is refused for dept B.

#	Issue	Where	Evidence / impact	How to fix
D3	Role trusted from the token	`security.py:create_access_token`	The role is signed into the JWT. If an admin demotes someone, the old token keeps the old role until it expires.	In `get_current_user`, always load the user from the DB and use `user.role` and `user.is_active`, not the token claim. Keep token lifetime short (for example 60 minutes).
D4	No login rate limiting	`routers/auth.py:173`	Unlimited password guesses per roll number. Roll numbers are easy to guess, so accounts are easy to brute-force.	Add `slowapi` (or a small in-memory counter for the demo): for example 5 attempts per minute per IP + identifier, return 429 after that.
D5	Placeholder room capacity treated as real	`services/timetable_import.py` (see contract line 46)	Rooms invented during import get capacity 40. The makeup solver then treats 40 as a true capacity and may book a 55-student section into it.	Store `capacity = None` for invented rooms (the solver already rejects unknown capacity), and return them in `rooms_created` so an admin fixes them with `PUT /api/rooms/{id}`.
D6	`create_staff_user` can 500 on duplicate	`routers/auth.py:238-269`	Check-then-insert without catching `IntegrityError`, unlike `register`. Two parallel requests with the same identifier give a 500.	Wrap `db.commit()` in `try/except IntegrityError` with `db.rollback()` and return 409, same as `register`.
D7	Uploads checked by extension only	`utils/file_storage.py:43`	A file named `x.pdf` with any content is accepted and later served to students.	Also check the first bytes (`%PDF-` for PDF, PNG/JPEG signatures for images) before saving, and serve downloads with `Content-Disposition: attachment`.
D8	`uploads/` is not in `.gitignore`	`.gitignore`, `utils/file_storage.py:3`	The docstring says uploads are "ignored by git"; they aren't. Student-uploaded papers can be committed by accident.	Add `backend/uploads/` and `*.db` to `.gitignore`.
D9	Every service function starts with `db.rollback()`	`services/swap_service.py`	It throws away any uncommitted work by the caller, silently. Safe now only because routers read `current_user.id` first (commented). Easy to break later.	Use `with db.begin():` (or a fresh session) inside the service instead of rolling back the caller's session, and document that services own their transaction.

Minor

- **Dev tools in production requirements:** `pytest` and `httpx` are in `requirements.txt`.
  - Fix: move them to `requirements-dev.txt` (`-r requirements.txt` + test tools).
- **Missing `__init__.py` files:** `routers/`, `services/`, `schemas/`, `utils/`, `core/` had them in #27 but they are gone now. Works as namespace packages, but some tools and IDEs misbehave.
  - Fix: add empty `__init__.py` to each package.
- **Test folder name:** the plan and #27 used `backend/tests/`; code is in `backend/test/`.
  - Fix: pick one (`tests/` is the pytest convention) and set `testpaths` in `pytest.ini`.
- **People's names in code and filenames:** "Wajiha's tests", "Sajawal's tables", `api-contract-wajiha.md`, `wajiha.postman_collection.json`.
  - Fix: describe the module, not the person; put ownership in `.github/CODEOWNERS` so GitHub asks the right reviewer automatically.
- **Stale comments:** "blocked until login is merged" (`permissions.py`), "When Taha merges core/permissions.py" (`portal_deps.py`), "Inline role check so this file has no dependency on Taha's permissions.py" (`auth.py`).
  - Fix: delete them as part of M6.
- **Hardcoded term:** `TimetableEntry.term` defaults to `"Fall 2026"`.
  - Fix: read the current term from settings (or a `terms` table) and require it on import.
- **Roll-number regex is a guess:** `^[a-z]{2,4}-\d{6,8}$` with example `Se-9244330`, while NUML emails use `numl-f24-12345`.
  - Fix: confirm the real roll-number format from the roster file and add 3 to 4 real examples as test cases.
- **Binary proposal in repo root:** `V-4_Timetable_Automation_Proposal_and_Team_Guide.docx`; it can't be diffed or reviewed.
  - Fix: move to `docs/` and keep a Markdown version for anything people need to change.
- **No backend README:** no run steps, env vars or test command.

    - Fix: add `backend/README.md`: venv, `pip install -r requirements-dev.txt`, copy `.env.example`, `alembic upgrade head`, `uvicorn app.main:app --reload`, `pytest`.
- **No linter or formatter:** style varies between authors.
    - Fix: add `ruff` (lint + format) with a `pyproject.toml` config and run it in the CI job from C7.

Work split

Each person fixes the code they wrote, so nobody has to learn a module from scratch. Phase 1 blocks everything else, so it goes first and in parallel.

Person	Phase 1: make it start (do first)	Phase 2: correctness and security	Phase 3: cleanup	Count
Sajawal (lead; wrote core, auth, models, makeup, swaps)	C1 `main.py`, C2 `config.py`, C3 `get_current_user` + `dependencies.py`, C5 `User` model vs `auth.py`	C6 `day_of_week` + teacher relationship, M3 lab rule, M4 makeup lock and ownership, M5 one rule engine, M8 register vs activate, D2 HOD department, D3 role from DB, D6 409 on duplicate, D9 service transactions	hardcoded term, roll-number regex, `__init__.py` files	16
Wajiha (wrote portal modules, import, tests)	C4 `from app.database` in 12 files + `pytest.ini`	M1 events PUT, M2 `instructor_name` rule, M6 one `require_roles`, M7 Alembic, M9 single API contract, D5 placeholder capacity, D7 file content check	`test/` to `tests/`, names out of code + `CODEOWNERS`, stale comments (with M6)	11

Person	Phase 1: make it start (do first)	Phase 2: correctness and security	Phase 3: cleanup	Count
Eisha	C7 CI workflow (can start now; turns green once Phase 1 lands)	D1 tests for auth, swaps, notifications, permissions, conflict checker; D4 login rate limit; D8 `.gitignore`	`requirements-dev.txt`, backend README, `ruff`, move proposal `.docx`	8

Hand-offs to watch:

1. Sajawal's C3 and Wajiha's C4 unblock everyone. Nobody starts Phase 2 until `pytest` runs on `main`.
2. Eisha writes D1 tests against the contract Wajiha writes in M9, so M9 should be drafted early in Phase 2. Failing tests she finds go back to the module's author, not fixed by her.
3. M5 (Sajawal) and M6 (Wajiha) both touch `permissions.py` and `conflict_checker.py`. Agree who merges first to avoid conflicts.
4. Every fix ships with its test, through a PR with CI green and one reviewer who ran it locally (C7).

Three things I'd push back on in this split:

- **Sajawal carries 16 of 35 items.** That's unavoidable for Phase 1 (it's his code), but if he's also integrating the frontend, Phase 2 will slip. If it does, M7 and D9 are the first to hand to Wajiha.
- **Taha isn't in this split.** The team guide makes him owner of permissions, conflict checker, swaps and notifications. If he's still on the team, C6, M5, D1-swaps and D9 belong to him, not Sajawal. Confirm this before assigning.
- **Eisha is a frontend member in the plan.** Her planned job (Vite, React Router, Axios client, `AuthContext`) isn't done; the `frontend` branch is plain HTML. Her backend items here are deliberately tests and tooling, which teach her the API she will call. Don't give her more backend work than this, or the frontend foundation slips further.

No deadlines are set here because none were given. Put a date on each phase in the team chat today; without one, Phase 1 drifts and nothing else can start.